

UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE ESTADÍSTICA

Máster en Big Data, Data Science e Inteligencia Artificial



UNIVERSIDAD
COMPLUTENSE
MADRID

Trabajo Fin de Máster

Sistema de visión artificial para detección y clasificación de vehículos militares

Alumno: Pablo Pérez Calvo

Madrid, Septiembre de 2025

Índice

1. Introducción	2
1.1. Motivación	2
1.2. Visión artificial en el sector militar	2
1.3. Objetivos	3
2. Estado del Arte	3
2.1. Modelos de detección de objetos	3
2.2. Evolución de YOLO	4
3. Preparación del dataset	5
3.1. Creación del dataset	5
3.2. Descripción del dataset	7
4. Implementación del modelo	10
4.1. Ajuste de hiperparámetros	10
4.2. Entrenamiento final del modelo	12
5. Evaluación y resultados	15
5.1. Ejemplos de predicción	17
6. Conclusiones	19
Bibliografía	20
A. Anexos	21
A.1. Creación del Dataset	21
A.2. Implementación del modelo	24

1. Introducción

1.1. Motivación

En el mundo actual, las técnicas de Inteligencia Artificial (IA) han revolucionado la forma en la que abordamos los desafíos tecnológicos, nos ha permitido automatizar tareas complejas y analizar grandes bases de datos para obtener información valiosa a través de ellos. La visión por computador es un campo de la IA que utiliza redes neuronales convolucionales (CNN) para obtener información significativa a través de imágenes o vídeos y poder realizar tareas como la detección, segmentación y clasificación de objetos.

La visión artificial puede ser aplicada en diversos sectores: en medicina, para la detección de lesiones y enfermedades a través de resonancias o radiografías; en movilidad urbana, para el conteo del flujo de vehículos para evitar la congestión del tráfico; y en ganadería, para automatizar la supervisión del número de animales y detectando ausencia de estos.

En este trabajo se aplicará un algoritmo de visión por computador al sector militar, con el objetivo de detectar e identificar distintos tipos de vehículos, tanto aéreos como terrestres. El sector militar es un sector crítico y estratégico que necesita eficiencia y seguridad en la detección de vehículos enemigos debido a la vital importancia del sector en la defensa el país.

1.2. Visión artificial en el sector militar

La Inteligencia Artificial ha cogido fuerza en el sector militar durante los últimos años , implementando numerosos sensores en soldados, drones, aviones, barcos, etc. con el objetivo de disponer de toda la información posible para tomar decisiones estratégicas. Estos sensores generan una gran cantidad de datos en tiempo real, lo que permite extraer patrones e información que sería imposible obtener a través de métodos tradicionales.

Los algoritmos de procesamiento de imágenes permiten crear sistemas de vigilancia avanzados, muy útiles en el ámbito de la defensa y seguridad porque permiten la detección temprana de amenazas e identificación de objetivos. Mediante cámaras, drones o imágenes satelitales se mantiene el control de los objetos existentes en la zona aliada y analizar la zona enemiga, diferenciar entre vehículos hostiles y aliados para evitar el fuego amigo, etc.

A su vez, las técnicas de visión artificial son necesarias para el desarrollo de sistemas de conducción autónoma y vehículos no tripulados [1]. El conocimiento completo del entorno permite a los vehículos conocer la posición de los objetos en todo momento, esto combinado con algoritmos de planificación de rutas, evitación de obstáculos y fusión de sensores permiten una conducción autónoma eficaz.

Un ejemplo de la aplicación de la inteligencia artificial en el ámbito militar es el tanque autónomo de 12 toneladas del ejército de EEUU, denominado como “RACER Heavy Platform”.



Figura 1: Tanque autónomo RACER Heavy Platform

1.3. Objetivos

El objetivo principal de este trabajo es desarrollar un algoritmo de visión por computador que obtenga métricas válidas para poder ser utilizado en un sistema de vigilancia de vehículos militares, en el campo de la defensa y seguridad del país.

Los objetivos específicos son:

- Recopilar y preparar una base de datos de imágenes de distintos tipos de vehículos militares, con el etiquetado adecuado.
- Entrenar un modelo capaz de detectar vehículos en imágenes y vídeos.
- El modelo deberá ser capaz de identificar el tipo de vehículo al que corresponde
- El modelo deberá ser capaz de realizar un seguimiento del movimiento del vehículo a lo largo de secuencias de vídeo.

2. Estado del Arte

2.1. Modelos de detección de objetos

La detección de objetos es una tarea de *Computer Vision* que ha evolucionado significativamente a lo largo de los años, especialmente con el avance de las técnicas de *Deep Learning*. La revolución de este campo fue causada por la aparición de las redes neuronales convolucionales (CNN), que permitieron aprender automáticamente jerarquías de características a

partir de los datos, mejorando notablemente la detección de objetos.

En 2014 surgieron los detectores de dos pasos, como *R-CNN*, que se encargaban de determinar regiones de interés en una imagen, pasaban esas regiones por la *CNN* y mediante un clasificador binario validaban si eran de clases correctas. Posteriormente surgen *Fast R-CNN* y *Faster R-CNN* para mejorar la propuesta inicial. El primero mejoraba el *IOU* y el *Loss* del posicionamiento de la *bounding box* y el segundo consiguió una mejora de velocidad [2]. Más tarde, *Mask R-CNN* permitió la segmentación de objetos, es decir, determinar la forma del mismo.

Los detectores de dos pasos eran muy precisos pero muy lentos. Es por ello que en 2016 se crean los detectores de un paso como SSD (*“Single Shot Multibox Detector”*) y YOLO (*“You Only Look Once”*).

2.2. Evolución de YOLO

YOLO es un detector que permite el procesamiento de imágenes en tiempo real y logra una velocidad extremadamente alta en la detección de objetos debido a que solo hace una única pasada a la red convolucional, prediciendo directamente las cajas limitadoras y las clases a las que corresponden los objetos. Presenta una arquitectura que procesa la imagen al completo durante el entrenamiento, permitiéndole aprender información sobre las clases de los objetos y su apariencia.

El modelo funciona dividiendo una imagen en una cuadrícula, en la que cada parte es responsable de detectar objetos en su área respectiva, se realiza múltiples predicciones para cada sección y se queda con las más precisas.

Desde YOLOv1 (2016) hasta YOLOv12 (2025), los modelos han evolucionado mejorando la precisión, la velocidad y la facilidad de uso. YOLOv1 utilizaba una CNN con 24 capas convolucionales y 2 totalmente conectadas. En versiones posteriores se adoptó la arquitectura Darknet como backbone, y se fueron introduciendo mejoras como multi-escala y atención espacial[3].

YOLOv11, versión que se va a utilizar en este trabajo para la detección de vehículos militares, incorpora bloques C3K2 (*“Cross Stage Partial with kernel size 2”*) sustituyendo al bloque C2F de anteriores versiones para mejorar la velocidad, SPPF (*“Spatial Pyramid Pooling - Fast”*), y C2PSA (*“Convolutional block with Parallel Spatial Attention”*). YOLOv11 consigue mejorar la precisión de YOLOv8 utilizando menos parámetros. [3].

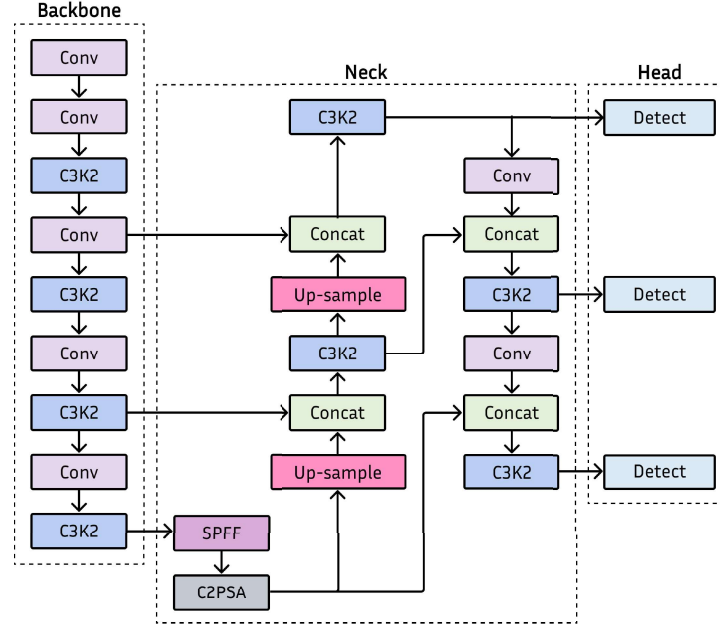


Figura 2: Arquitectura de YOLOv11

Ultralytics [4] es una empresa que desarrolla implementaciones optimizadas de los modelos YOLO en *PyTorch*. Su librería en *Python* facilita el uso de estos modelos permitiendo el entrenamiento con conjuntos de datos personalizados, carga de modelos preentrenados y la obtención de predicciones de manera eficiente. En este trabajo se empleará dicha librería para implementar YOLOv11 para la detección de vehículos militares.

3. Preparación del dataset

3.1. Creación del dataset

El dataset definitivo que se ha utilizado en este proyecto ha sido construido a partir de la combinación de varios datasets públicos de RoboFlow.

En primer lugar, se ha tomado como dataset base el *Military Vehicle Detection Dataset* [5]. Un dataset que contiene más de 9000 imágenes y cuenta con las siguientes clases:

```
Nombre de las clases: ['Armored_car', 'Car', 'Person', 'RSZO', 'Tank', 'Truck', 'apc', 'army-truck', 'artillery-gun', 'car', 'hummer', 'military-vehicle', 'person', 'pickup', 'rocket-artillery', 'soldier', 'tank', 'truck', 'van', 'vehicle']
```

```
Tank: 3462 objetos, rocket-artillery: 210 objetos, military-vehicle: 1856 objetos, Person: 4980 objetos, Armored_car: 199 objetos, Car: 1821 objetos, army-truck: 560 objetos, car: 2302 objetos, tank: 681 objetos, hummer: 309 objetos, apc: 416 objetos, person: 199 objetos, van: 52 objetos, RSZO: 308 objetos, Truck: 41 objetos, pickup: 28 objetos, soldier: 34 objetos, truck: 1 objetos, vehicle: 38 objetos, artillery-gun: 3 objetos
```

Como podemos observar, este dataset tiene varias clases poco representadas, algunas con errores en la escritura o que se refieren a la misma clase de vehículo y otras que no son relevantes para el objetivo del trabajo. Para solucionar este problema se hará una selección y unificación de clases, priorizando aquellas que correspondan a vehículos militares y tengan una representación considerable.

- Se definió la clase *Tank*, agrupando las etiquetas *Tank* y *tank*.
- Se creó la clase *rocket_launcher*, agrupando *RSZO* y *rocket-artillery*.
- Se creó la clase *military_truck*, agrupando *truck*, *Truck*, *army-truck* y *hummer*.

La clase *military-vehicle* no ha sido tenido en cuenta a pesar de su gran representación porque era de carácter general y queremos que el modelo llegue a identificar el tipo de vehículo.

Se han reindexado las clases mencionadas anteriormente y las etiquetas de las clases no seleccionadas han sido eliminadas. Esto provoca la existencia de imágenes sin anotaciones que han sido eliminadas.

Una vez hecho este primer preprocesamiento el dataset base cuenta con 4353 imágenes que contienen en ella 4143 *tanks*, 518 *rocket launchers* y 911 *military trucks*. Se ha optado por añadir más imágenes de otros datasets públicos de Roboflow con el objetivo de enriquecer la base de datos, aumentando el número de clases y el número de objetos etiquetados en ellas:

- El dataset [6] ha sido utilizado para añadir las clases *military_aircraft* y *military_helicopter* y aumentar el número de imágenes y etiquetas de *military_truck*.

- El dataset [7] ha sido utilizado para aumentar el número de imágenes y etiquetas de las clases *rocket_launcher* y *military_truck*.
- El dataset [8] ha sido utilizado para aumentar el número de imágenes y etiquetas de la clase *military_aircraft*.
- El dataset [9] ha sido utilizado para aumentar el número de imágenes y etiquetas de la clase *military_helicopter* (mayoritariamente) y de las clases *military_truck* y *military_aircraft*.

Tras añadir los datasets comentados anteriormente a nuestro dataset principal, se dispone de 11745 imágenes.

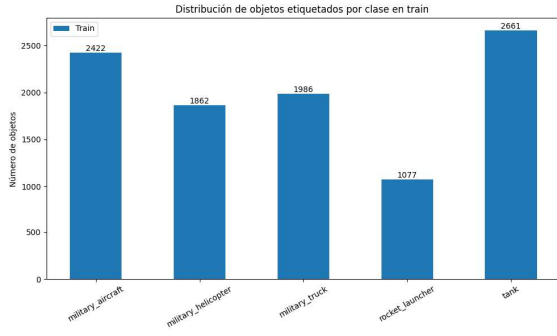
Finalmente, tras haber llevado a cabo un proceso de análisis exhaustivo en la plataforma de Roboflow con el fin de mejorar la calidad y consistencia del dataset.

- Se han reetiquetado imágenes con anotaciones incorrectas.
- Se han etiquetado objetos sin anotación en algunas imágenes.
- Se han eliminado imágenes de baja calidad y poco representativas para el problema.
- Se han añadido selectivamente imágenes de otros datasets públicos de Roboflow para reforzar clases con poca representación.

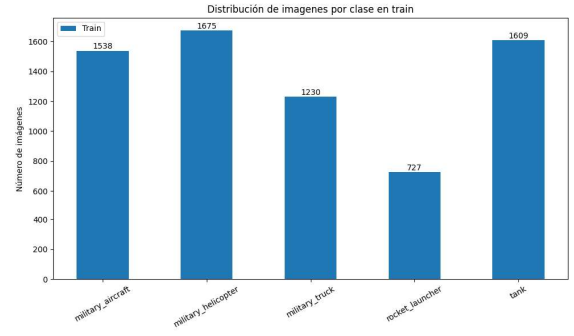
3.2. Descripción del dataset

Una vez realizado el proceso de construcción del dataset descrito en la sección anterior, el dataset definitivo [10] dispone de 8487 imágenes, distribuidas de la siguiente manera: 6579 para *train* (77%), 1174 para *valid* (14%) y 734 para *test* (9%).

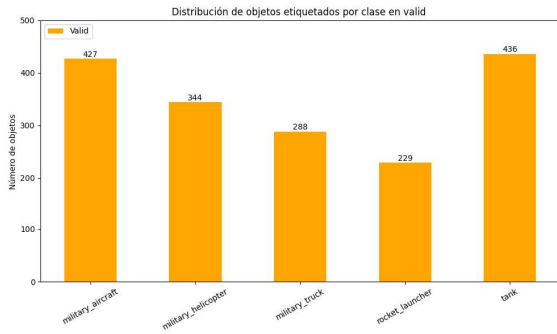
La distribución de objetos etiquetados y número de imágenes en las que aparece cada clase en cada subconjunto se puede observar en los siguientes gráficos:



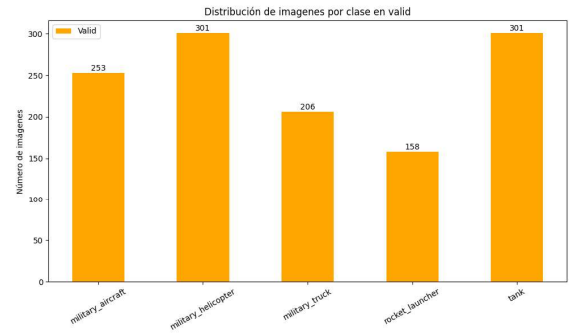
(a) Objetos etiquetados por clase en train



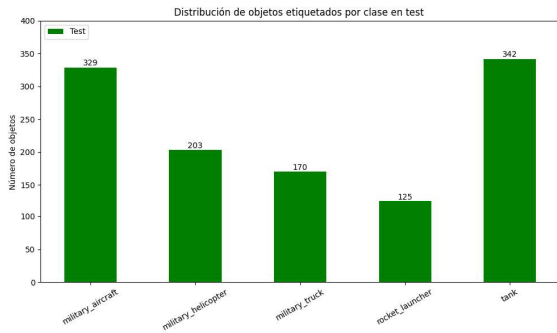
(b) Imágenes por clase en train



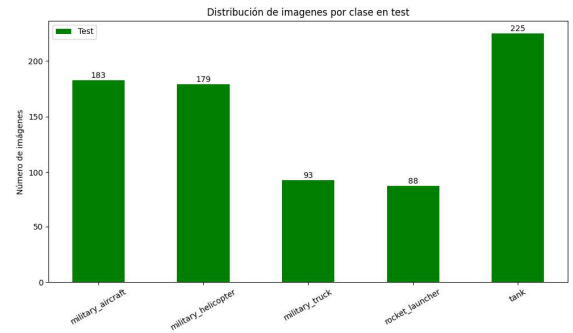
(c) Objetos etiquetados por clase en valid



(d) Imágenes por clase en valid



(e) Objetos etiquetados por clase en test



(f) Imágenes por clase en test

Figura 3: Distribución del número de objetos etiquetados y el número de imágenes en el que aparece algún objeto por clase.

Se puede observar que la clase *rocket_launcher* tiene poca representación comparada con el resto, esto es debido a que en escenarios reales son objetos raros comparado con tanques, aviones, helicópteros o camiones.

Nuestro dataset presenta imágenes de 5 clases de vehículos distintas, 3 correspondientes a vehículos terrestres y 2 a vehículos aéreos. Aquí se pueden ver algunos ejemplos.



(a) Military_aircraft



(b) Military_helicopter



(c) Tank



(d) Military_truck



(e) Rocket_launcher

Figura 4: Ejemplo de imágenes de cada una de las clases.

4. Implementación del modelo

4.1. Ajuste de hiperparámetros

En primer lugar, se ha realizado un ajuste de hiperparámetros con la función *model.tune* de la librería *Ultralytics* [4] para optimizar el posterior entrenamiento del modelo. Esto se ha llevado a cabo realizando 10 iteraciones, cada una evaluada durante 30 épocas, explorando distintas combinaciones de valores en 16 hiperparámetros como el *learning_rate*, *translate* y *mosaic*, entre otros A.2.

La métrica que utiliza la función *model.tune* para determinar que combinación de parámetros es mejor se trata del *fitness*. En detección de objetos, la función calcula el *fitness* como una suma ponderada de *Precision*, *Recall*, *mAP@50*, *mAP@50-95*, dándole mayor peso a estas dos últimas. La evolución del *fitness* en cada iteración y los resultados del ajuste de hiperparámetros realizado, se pueden observar en las figuras 5 y 6.

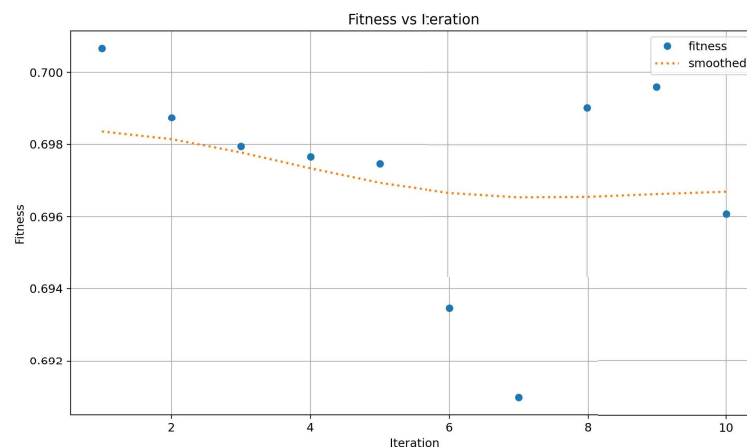


Figura 5: Evolución del fitness en cada iteración.

fitness	lr0	lrf	momentum	weight_decay	warmup_epochs	warmup_momentum	box	cls	dfl	hsv_h	hsv_s	hsv_v	translate	scale	flipr	mosaic
0.7007	0.0100	0.0100	0.9370	0.0005	3.0000	0.8000	7.5000	0.5000	1.5000	0.0150	0.7000	0.4300	0.1000	0.5000	0.5000	1.0000
0.6987	0.0082	0.0126	0.9638	0.0005	3.0000	0.8103	8.9668	0.5092	1.3461	0.0118	0.9000	0.4300	0.0715	0.4902	0.4688	0.9574
0.6979	0.0100	0.0095	0.9370	0.0006	2.9387	0.8000	7.8178	0.5000	1.2823	0.0162	0.7000	0.3386	0.1035	0.4431	0.5231	1.0000
0.6977	0.0095	0.0107	0.9220	0.0005	3.2706	0.8330	7.1083	0.5105	1.4963	0.0157	0.6505	0.4337	0.0970	0.4999	0.5117	1.0000
0.6975	0.0100	0.0100	0.9377	0.0005	3.0472	0.7899	7.6685	0.4987	1.5233	0.0149	0.7067	0.4356	0.0991	0.5000	0.5062	0.9999
0.6935	0.0100	0.0081	0.9227	0.0005	3.0000	0.7442	7.5000	0.4324	1.1972	0.0146	0.7916	0.4762	0.1133	0.5584	0.5870	0.9392
0.6910	0.0100	0.0100	0.9694	0.0005	2.2280	0.8179	6.0509	0.4141	1.2795	0.0141	0.8157	0.3500	0.1007	0.4722	0.6139	0.8125
0.6990	0.0082	0.0126	0.9579	0.0005	3.0000	0.8302	8.9602	0.5074	1.3262	0.0117	0.8826	0.3952	0.0718	0.4892	0.4674	0.9574
0.6996	0.0097	0.0098	0.9370	0.0005	2.8602	0.8441	7.8366	0.5308	1.5000	0.0146	0.7339	0.4000	0.0962	0.4893	0.5000	1.0000
0.6961	0.0090	0.0159	0.9559	0.0006	2.9273	0.6549	8.9668	0.4965	1.4179	0.0105	0.8533	0.3676	0.0562	0.4644	0.4153	0.9574

Created with Datawrapper

Figura 6: Resultados del ajuste de hiperparámetros durante 10 iteraciones

La mejor combinación de hiperparámetros y por tanto, la que se utilizará para entrenar el modelo es la siguiente.

```
# 10/10 iterations complete      (13861.04s)
# Best fitness=0.70066 observed at iteration 1
# Best fitness hyperparameters are printed below.

lr0: 0.01
lrf: 0.01
momentum: 0.937
weight_decay: 0.0005
warmup_epochs: 3.0
warmup_momentum: 0.8
box: 7.5
cls: 0.5
dfl: 1.5
hsv_h: 0.015
hsv_s: 0.7
hsv_v: 0.4
degrees: 0.0
translate: 0.1
scale: 0.5
shear: 0.0
perspective: 0.0
flipud: 0.0
fliplr: 0.5
bgr: 0.0
mosaic: 1.0
mixup: 0.0
cutmix: 0.0
copy_paste: 0.0
```

4.2. Entrenamiento final del modelo

Una vez obtenida la configuración de hiperparámetros óptima, se procedió al entrenamiento definitivo, utilizando un modelo YOLOv11 de la librería *Ultralytics* [4]. El entrenamiento se ejecutó durante 200 épocas, utilizando imágenes de tamaño 640X640 píxeles y un tamaño de lote de 16 A.2.

Para reducir el riesgo de sobreajuste se han usado estrategias de regularización como la técnica de *early stopping* con una paciencia de 30 épocas y técnicas de *data augmentation* como *mosaic*, *translate* y *flip* incluidas en la configuración de los hiperparámetros.

Inicialmente se entrenó un modelo YOLOv11s una versión ligera de la arquitectura que permitió conocer el funcionamiento del modelo con el dataset. Posteriormente, se entrenó un modelo YOLOv11m, el cual es una versión que mantiene un equilibrio intermedio entre precisión y coste computacional. Los resultados obtenidos en las métricas de validación con este modelo fueron superiores a los del primero, por lo que utilizó el modelo entrenado con YOLOv11m para los experimentos finales.

El proceso se ha llevado a cabo en el entorno de Google Colab Pro, usando una GPU NVIDIA A100, lo que aceleró significativamente el entrenamiento. El entrenamiento terminó en la época 178 y el tiempo de ejecución fue de aproximadamente 3 horas, antes de lo previsto debido a la técnica de *early stopping*.

Antes de analizar los resultados del entrenamiento, definiremos las métricas que ofrece el entrenamiento de un modelo YOLO :

- **Precision:** Mide la proporción de predicciones positivas que realmente fueron correctas.
- **Recall:** Mide la proporción de positivos reales que se midieron correctamente, es decir, indica si el modelo es capaz de encontrar todos los objetos de la clase. También es conocida como sensibilidad.
- **mAP50:** Mide la precisión media en un umbral de IoU de 0,5. Esta métrica permite comprobar si el modelo es capaz de encontrar y localizar los objetos correctamente con un requisito de precisión menos estricto.
- **mAP50-95:** Mide la precisión media promedio en múltiples umbrales de IoU, desde 0.5 hasta 0.95 en incrementos de 0.05. Esta métrica nos ofrece una visión más completa de la precisión con la que el modelo puede encontrar objetos.

La siguiente gráfica nos muestra la evolución en cada época de las distintas métricas que ofrece YOLO durante el entrenamiento.

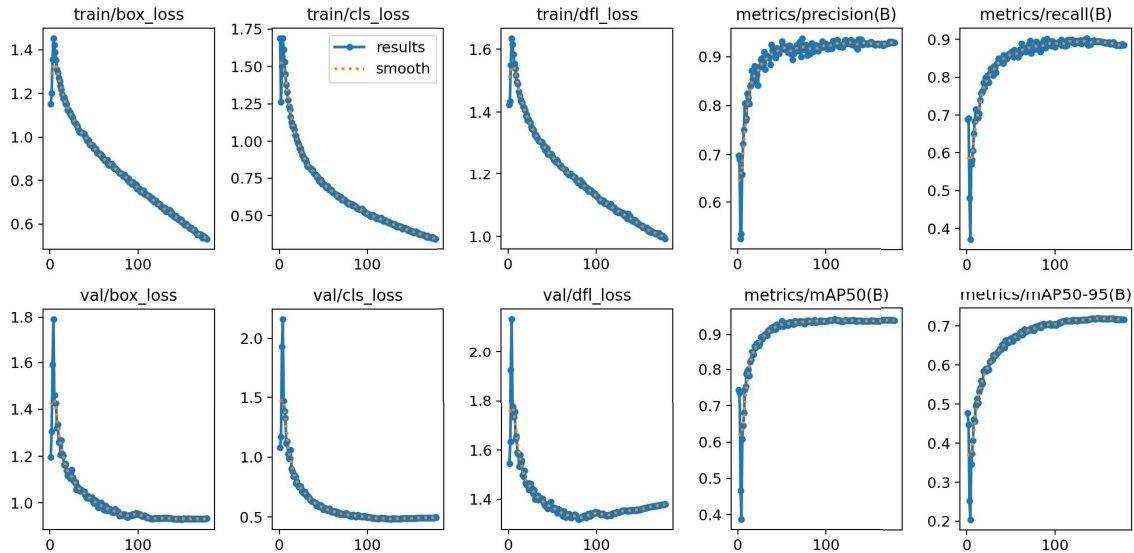


Figura 7: Gráfica de la evolución de las métricas durante el entrenamiento.

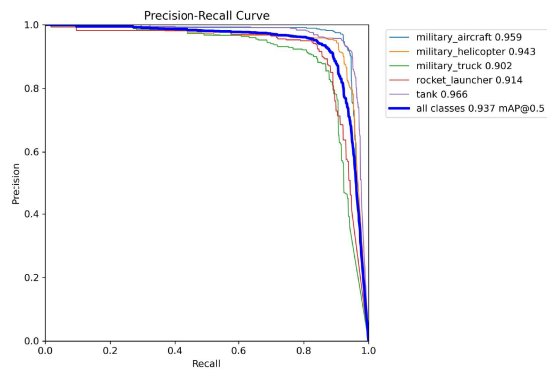
Podemos observar en las gráficas de *loss* en validación, como decrece rápidamente en las primera épocas y luego tiende a estabilizarse. Esto se debe a que, en las últimas etapas del entrenamiento cada vez es más difícil mejorar significativamente el rendimiento del modelo. Aunque las pérdidas de validación tienden a estabilizarse, convergen hacia valores muy similares a los de entrenamiento, por lo que no existe un sobreajuste significativo.

En cuanto a las gráficas de las métricas de *precisión*, *recall*, *mAP50* y *mAP50-95*, podemos ver una evolución creciente y constante, que se estabiliza en valores altos sin caídas bruscas, lo que indica un buen rendimiento del modelo.

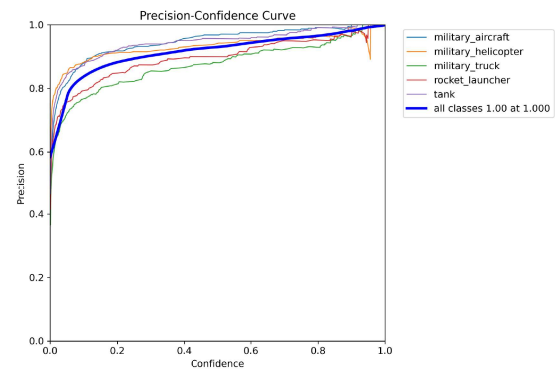
Como el objetivo de nuestro problema es detectar vehículos en el sector militar, siendo este un sector crítico, necesitamos que nuestro modelo presente buen rendimiento equilibrado en precisión y recall. Para nuestro problema es igual de importante identificar bien el tipo de vehículo, al mismo tiempo que sepa detectar bien vehículos.

La métrica **F1-Score** es la que se encarga de optimizar este equilibrio, ya que es calculada con una media armónica de los valores de precisión y recall. Esta métrica sirve para evitar falsos análisis de resultados, un modelo con buena precisión no quiere decir que tenga buen rendimiento.

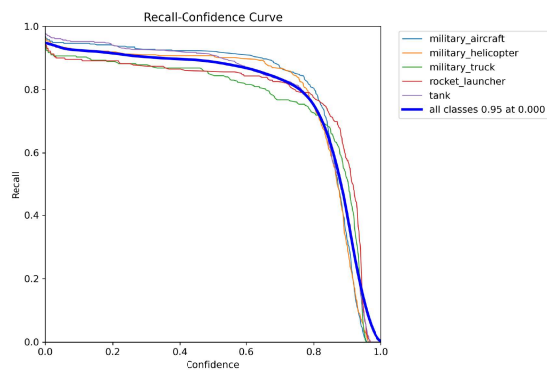
En la figura 8 podemos analizar como se comporta el modelo para distintos valores del umbral de confianza.



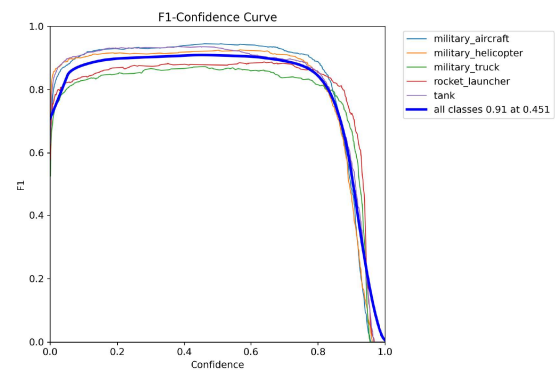
(a) Precision - Recall Curve



(b) Precision - Confidence Curve



(c) Recall - Confidence Curve



(d) F1 - Confidence Curve

Figura 8: Gráficos de rendimiento del modelo

Podemos ver que a medida que aumenta el umbral de confianza que el modelo asigna en una detección, la precisión aumenta y el recall disminuye. Esto es un comportamiento lógico porque al aumentar la confianza se reduce los falsos positivos y se dejan de detectar algunos objetos. Además, el modelo alcanza valores altos en precisión y recall, manteniendo un buen equilibrio.

Si nos fijamos en la gráfica de *F1-Confidence* tenemos que el umbral óptimo de confianza en el que nuestro modelo obtiene el máximo valor de *F1* es en 0,451. Como esta métrica es muy importante para nuestro objetivo, usaremos ese umbral de confianza para futuras validaciones y predicciones.

Los resultados que se obtuvieron sobre el conjunto de validación durante el entrenamiento vienen recogidos en la siguiente tabla.

Class	Images	Instances	P	R	mAP50	mAP50-95
all	1174	1724	0.93	0.894	0.926	0.742
military_aircraft	253	427	0.966	0.923	0.954	0.761
military_helicopter	301	344	0.94	0.907	0.93	0.681
military_truck	206	288	0.877	0.868	0.9	0.729
rocket_launcher	158	229	0.912	0.856	0.901	0.764
tank	301	436	0.955	0.915	0.947	0.776

Tabla 1: Resultados sobre el conjunto de validación durante el entrenamiento.

Los resultados de la tabla 1 muestran que el modelo es bastante sólido, presentando unos muy buenos resultados para la evaluación global de todas las clases en el entrenamiento, con un *mAP50* de 0,926 y un *mAP50-95* de 0,742. Las clases *military_aircraft* y *tank* son las que tienen resultados más consistentes. La clase *military_truck* es la más débil aunque con buenos resultados, probablemente por ser la clase con mayor variabilidad de formas y tipos de vehículos.

5. Evaluación y resultados

Para obtener una visión más general del rendimiento de nuestro modelo, vamos a evaluarlo en nuestro conjunto de prueba, un conjunto de imágenes que no han sido utilizadas durante el entrenamiento.

Class	Images	Instances	P	R	mAP50	mAP50-95
all	734	1168	0.915	0.852	0.901	0.711
military_aircraft	183	329	0.903	0.848	0.884	0.697
military_helicopter	179	203	0.94	0.931	0.953	0.708
military_truck	93	169	0.827	0.817	0.865	0.664
rocket_launcher	88	125	0.945	0.832	0.896	0.796
tank	225	342	0.959	0.83	0.906	0.689

Tabla 2: Resultados del modelo sobre el conjunto de prueba

En la tabla 2 podemos observar como el rendimiento baja ligeramente respecto a validación, un comportamiento esperable debido a que se están evaluando imágenes que no se han tenido cuenta en el entrenamiento. Aún así, las métricas que nos ofrece el modelo son muy buenas y la disminución es muy leve, lo que nos indica que el modelo tiene buena capacidad

de generalización.

La clase *military_truck* sigue siendo la de menor rendimiento en la evaluación. La clase *rocket_launcher* tiene grandes métricas a pesar de ser la clase menos representada. Finalmente, las clases *military_aircraft*, *military_helicopter* y *tank* son estables y ofrecen muy buenos resultados.

La matriz de confusión normalizada de los resultados del modelo en el conjunto *test* es la siguiente:

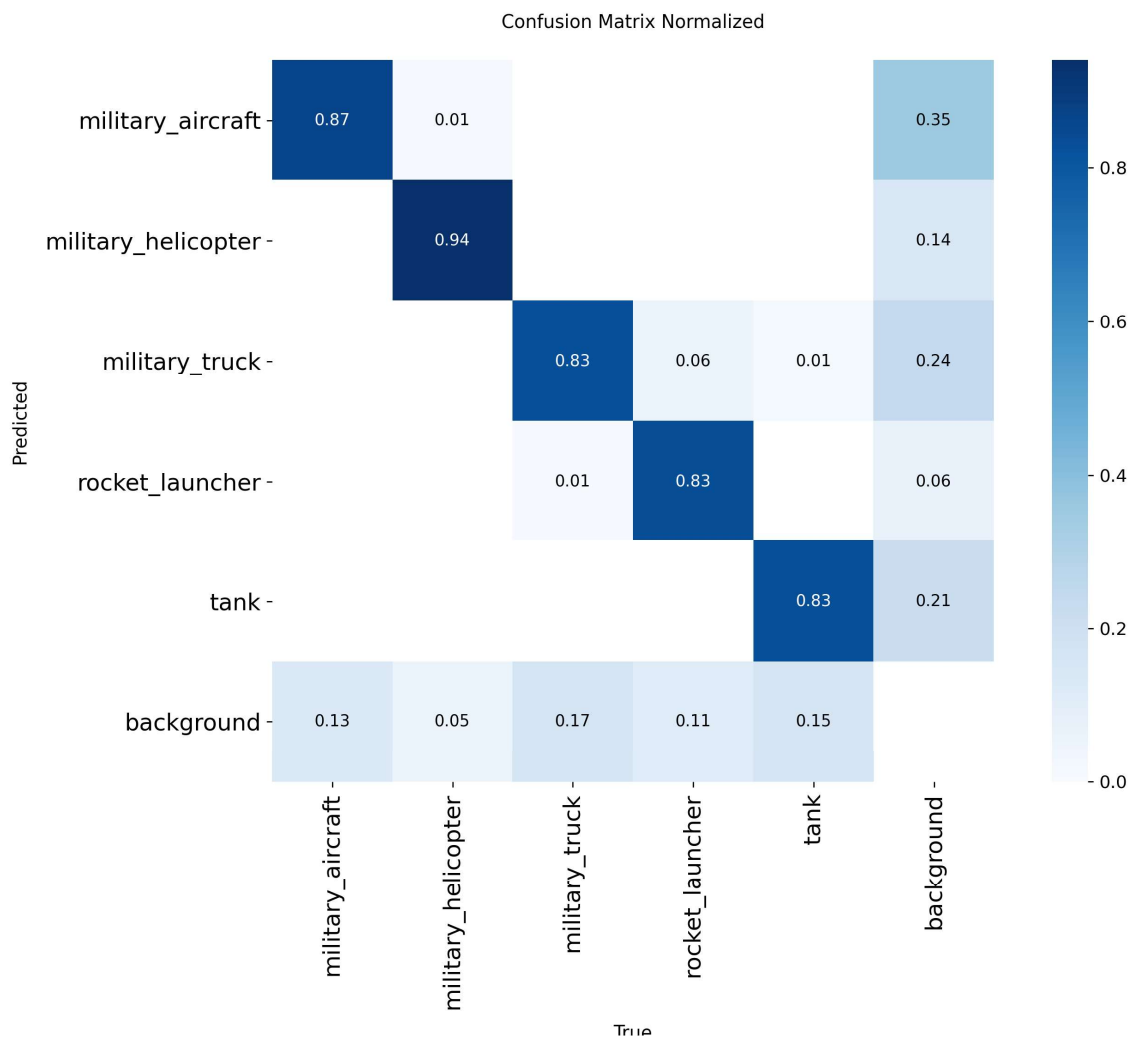


Figura 9: Matriz de confusión normalizada

El modelo presenta un rendimiento adecuado en la clasificación, la confusión entre clases es casi inexistente. La confusión más frecuente se da entre *military_truck* y *rocket_launcher*, debido a su similitud en el tipo de vehículo. Hay que destacar que existe un pequeño problema en la detección de *background* incorrectamente. Esto se podría mejorar en futuros trabajos

añadiendo imágenes sin anotaciones con fondos característicos del sector militar.

5.1. Ejemplos de predicción

Ahora vamos a mostrar algunos ejemplos de predicciones que ha realizado el modelo sobre imágenes del conjunto *test*.



Figura 10: Ejemplos de predicción de vehículos

Además de los buenos resultados con imágenes donde solo hay un tipo de vehículo, el modelo también es capaz de distinguir el tipo de vehículo en una misma imagen.



Figura 11: Ejemplos de imágenes con varias clases

6. Conclusiones

Con la realización de este trabajo se ha conseguido implementar un algoritmo de visión por computador que permite detectar e identificar vehículos militares con alta precisión. Los objetivos que se habían planteado en la primera sección se cumplieron con éxito.

La base de datos fue creada a partir de una fusión de distintos datasets públicos, se realizó un preprocesamiento adecuado del dataset que fue clave para la obtención de un modelo robusto

Se realizó un ajuste de hiperparámetros con la intención de obtener mejores resultados en el entrenamiento del modelo. El entrenamiento se ha realizado con un modelo YOLO11 utilizando la librería Ultralytics y el modelo ha sido capaz de detectar vehículos militares en imágenes y videos obteniendo grandes resultados que indican un buen rendimiento.

El modelo ha mostrado grandes métricas en la predicción de cada clase, presentando pocos errores de confusión entre ellas. Esto nos permite conocer la calidad del modelo en la identificación del tipo de vehículo, siendo capaz de distinguir diferentes tipos de vehículos en la misma imagen.

La principal limitación de este proyecto ha sido no disponer de un equipo con GPU. Para el entrenamiento se utilizó Google Colab Pro, lo que permitió acceso a una GPU NVIDIA A100 pero con restricciones en la cantidad y tiempo de uso. Disponer de un equipo con GPU habría permitido aumentar el número de imágenes del dataset para mejorar el rendimiento y generalización del modelo.

Como recomendación para futuros trabajos, se podría incluir imágenes de paisajes característicos del sector militar sin anotaciones, lo que ayudaría a mejorar el *recall* y disminuir la detección de falsos positivos. Además, sería interesante añadir otros tipos de vehículos e incluso, personal militar para aumentar el realismo del problema.

Referencias

- [1] Rodrigo Álvarez. Desarrollo y uso de nuevas tecnologías e inteligencias artificiales en el ámbito militar, 2019.
- [2] Oleksandr Barmak1 Dmytro Borovyk1 and Volodymyr Lytvynenko. Detection of military targets from images using deep learning models, apr 2025.
- [3] Leo Thomas Ramos and Angel D. Sappa. A decade of you only look once (yolo) for object detection: A review, apr 2025.
- [4] Ultralytics documentation. <https://docs.ultralytics.com/es/models/yolo11>.
- [5] oDec. Military vehicle detection dataset. <https://universe.roboflow.com/odec/military-vehicle-detection-5mb9k>, mar 2024.
- [6] military. Military dataset. <https://universe.roboflow.com/military-sypqz/military-ni53h>, jan 2024.
- [7] Huyen Dinh. military-vehicle dataset. <https://universe.roboflow.com/huyen-dinh/military-vehicle-rstvv>, jul 2023.
- [8] flight. military aircraft dataset. <https://universe.roboflow.com/flight-gegaf/military-aircraft-0vcks>, apr 2025.
- [9] Military Helicopter. Military helicopter dataset dataset. <https://universe.roboflow.com/military-helicopter/military-helicopter-dataset>, may 2025.
- [10] Pablopercal. Dataset definitivo del tfm. https://universe.roboflow.com/pablopercal/tfm_dataset-tfgcw, sep 2025.

A. Anexos

En esta sección se mostrarán algunos códigos en *Python* que han sido útiles para la implementación del trabajo. El resto del código se encuentra en los archivos `.ipynb`.

A.1. Creación del Dataset

La siguiente función se encarga de leer un dataset en archivo zip. Los datasets que se han utilizado están organizados en tres carpetas: *train*, *valid* y *test*, dentro de cada una también están dividido en *images* y *labels*.

```
def lectura_dataset(ruta_origen, ruta_destino):

    with zipfile.ZipFile(ruta_origen, 'r') as zip:
        zip.extractall(ruta_destino)
```

La siguiente función ha sido utilizada para contar el número de objetos etiquetados por cada clase.

```
def Recuento_clase(ruta_labels):
    # Inicializamos contador
    contador = Counter()

    # Leer todos los archivos .txt
    for archivo in os.listdir(ruta_labels):
        if archivo.endswith(".txt"):
            ruta_archivo = os.path.join(ruta_labels, archivo)
            with open(ruta_archivo, 'r') as f:
                for linea in f:
                    # Leer el id de las clases en cada txt
                    id_clase = int(linea.strip().split()[0])
                    contador[id_clase] += 1

    return contador
```

La función *renombrar_labels* se ha utilizado para renombrar las etiquetas y unificar todas las que correspondían a la misma clase.

```
def renombrar_labels(ruta_labels, ruta_labels_destino, indice_mapeo):

    for archivo in os.listdir(ruta_labels):
        if not archivo.endswith(".txt"):
            continue

        ruta_origen = os.path.join(ruta_labels, archivo)
        ruta_destino = os.path.join(ruta_labels_destino, archivo)
```

```

# Lectura de cada archivo
with open(ruta_origen, 'r') as f:
    lineas = f.readlines()

lineas_nuevas = []
# Recorre cada linea del archivo y mira el primer elemento (la
clase)
for linea in lineas:
    texto = linea.strip().split()
    if not texto:
        continue
    clase_antigua = int(texto[0])
    # Comprueba si se debe reindexar y se renombra la clase
    if clase_antigua in indice_mapeo:
        clase_nueva = indice_mapeo[clase_antigua]
        linea_nueva = f"{clase_nueva} "+" ".join(texto[1:]) + "\n"
    else:
        lineas_nuevas.append(linea_nueva)

# Si no esta en el mapeo de indice la linea desaparecera
porque no queremos esa etiqueta
if lineas_nuevas:
    with open(ruta_destino, 'w') as f_destino:
        f_destino.writelines(lineas_nuevas)
    print(f"Etiquetas renombradas con exito, se ha guardado en: {
ruta_labels_destino}")

```

Aquí se muestra un ejemplo de como se ha realizado el proceso de agrupamiento y renombramiento de las clases del primer dataset. (Para el resto de datasets que se han utilizado se ha hecho un proceso análogo).

```

nueva_ruta_yaml = "/content/MilitaryVehicleDetection/data_reindexed.
yaml"
nombre_clases = lista_clases(ruta_yaml)
# Agrupar las clases que nos interesan
agrupamientos = {
    'Tank': 'tank',
    'tank': 'tank',
    'army-truck': 'military_truck',
    'hummer': 'military_truck',
    'RSZO': 'rocket_launcher',
    'rocket-artillery': 'rocket_launcher',

```

```

        'truck': 'military_truck',
        'Truck': 'military_truck',
    }

# Lista final de las nuevas clases ordenada
clases_agrupadas = sorted(list(set(agrupamientos.values())))
print("Clases definitivas:", clases_agrupadas)

# Creación del mapeo de índices para renombrar las clases
mapeo_indice = {}
for id_antiguo, nombre_antiguo in enumerate(nombre_clases):
    if nombre_antiguo in agrupamientos:
        nombre_nuevo = agrupamientos[nombre_antiguo]
        id_nuevo = clases_agrupadas.index(nombre_nuevo)
        mapeo_indice[id_antiguo] = id_nuevo

# Aplicamos el reetiquetado de las clases
renombrar_labels(labels_train, labels_train_destino, mapeo_indice)
renombrar_labels(labels_valid, labels_valid_destino, mapeo_indice)
renombrar_labels(labels_test, labels_test_destino, mapeo_indice)

# Guardar archivo YAML con las nuevas clases
nuevo_yaml = { 'train': '../train/images',
                'val': '../valid/images',
                'test': '../test/images',
                'nc': len(clases_agrupadas),
                'names': clases_agrupadas
            }
with open(nueva_ruta_yaml, 'w') as f:
    yaml.dump(nuevo_yaml, f)
print(f"Nuevo archivo YAML guardado en: {nueva_ruta_yaml}")

```

Al hacer una selección de clases, algunas imágenes que se quedaron sin etiquetas se borraron con la siguiente función.

```

def borrar_fotos_nulas(ruta_labels, ruta_imagenes):
    # Contador
    borradas = 0
    # Recorre todas las imagenes
    for imagen in os.listdir(ruta_imagenes):
        if imagen.endswith(("jpg", "jpeg", "png")):
            # Obtiene el nombre de la imagen
            nombre_archivo = os.path.splitext(imagen)[0]

```



```

archivo = nombre_archivo + ".txt"
ruta_label = os.path.join(ruta_labels, archivo)

# Si no existe el archivo txt, borra la imagen
if not os.path.exists(ruta_label):
    os.remove(os.path.join(ruta_imagenes, imagen))
    borradas += 1
print(f"Se eliminaron {borradas} imagenes sin etiquetas.")

```

A.2. Implementación del modelo

En primer lugar, cargamos el modelo YOLO de la librería Ultralytics.

```
model = YOLO('yolo11m.pt')
```

En segundo lugar, se ha utilizado el siguiente código para el ajuste de hiperparámetros.

```

model.tune(
    data="/content/TFM/data.yaml",
    epochs=30,
    batch=16,
    name="/content/drive/MyDrive/yolo_runs/tune",
    resume=True
)

```

Por último, se entrena el modelo cargando los hiperparámetros obtenidos anteriormente.

```

best_hyp = "/content/drive/MyDrive/yolo_runs/tune/
            best_hyperparameters.yaml"

model.train(
    data="/content/TFM/data.yaml",
    epochs=200,
    batch=16,
    imgsz=640,
    patience=30,
    save_period=10,
    project="/content/drive/MyDrive/yolo_runs",
    name="train_1909",
    cfg=best_hyp,
    seed=1909
)

```